Chapter 4

Local Search and Optimization Problems

Amit Kumar Mondal
Assistant Professor, CSE KU

Local Search and Optimization Problems

- ***Systematic search*** - keep one or more paths in memory + record which alternatives have been explored at each point along the path.
- *path* to that goal also constitutes a *solution*
- many problems, the path to the goal is irrelevant – 8 queens
- integrated-circuit design, factory-floor layout, job-shop scheduling, automatic programming

Local Search

- **Local search** algorithms operate using a single **current node** ---
  - move only to neighbors
  - Paths are not retained

State space = a set of complete configuration

Two key advantages:

- (1) use very little memory—usually a constant amount;
- (2) find reasonable solutions in large or infinite (continuous) state spaces for which systematic algorithms are unsuitable.
- Useful for solving pure ***optimization problems***, in which the aim is to find the best state according to an ***objective function*****.**

Hill Climbing

Algorithm:

1. Generate a possible solution.
2. Test to see if this is actually a solution.
3. Quit if a solution has been found.
   Otherwise, return to step 1.

- Keep a single “current” state, try to improve it.
- ■ Basic idea is to always head towards a state which is better than the current one.
- ■ Generate-and-test + direction to move.
- ■ Heuristic function to estimate how close a given state is to a goal state.
- ■ Considers all the moves from the current state. Pick the best of the bunch and replace the current state with that one.
- ■ It does not backtrack the search space, as it does not remember the previous states

Hill Climbing

- ■ The initial state is chosen at random!
- ■ The algorithm may return different solutions depending on the start
- ■ If the options do not improve the current state, remain stuck at the current state
- ■ It’s the main difference with Greedy Search.
- - In greedy search, always move to the next (best among the options)

Hill Climbing

- Let’s look at the Simple Hill climbing algorithm:
- 1. Define the current state as an initial state

- 2. Loop until the goal state is achieved or no more operators can be applied on the current state:

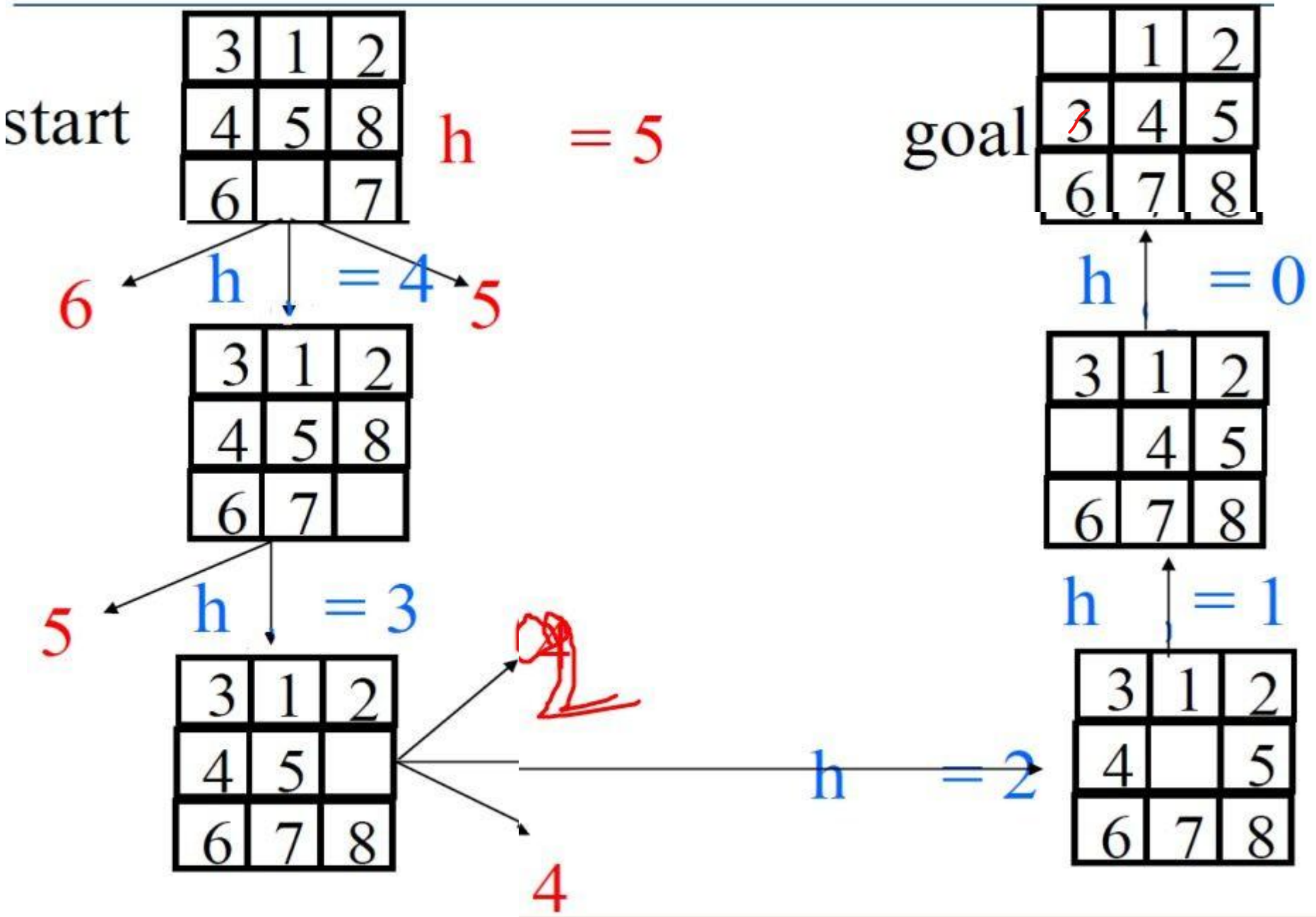
    - 2.1 Apply an operation to current state and get a new state
    - 2.2 Compare the new state with the goal
    - 2.3 Quit if the goal state is achieved
    - 2.4 Evaluate new state with heuristic function and compare it with the current state
    - 2.5 If the newer state is closer to the goal compared to current state, update the current state

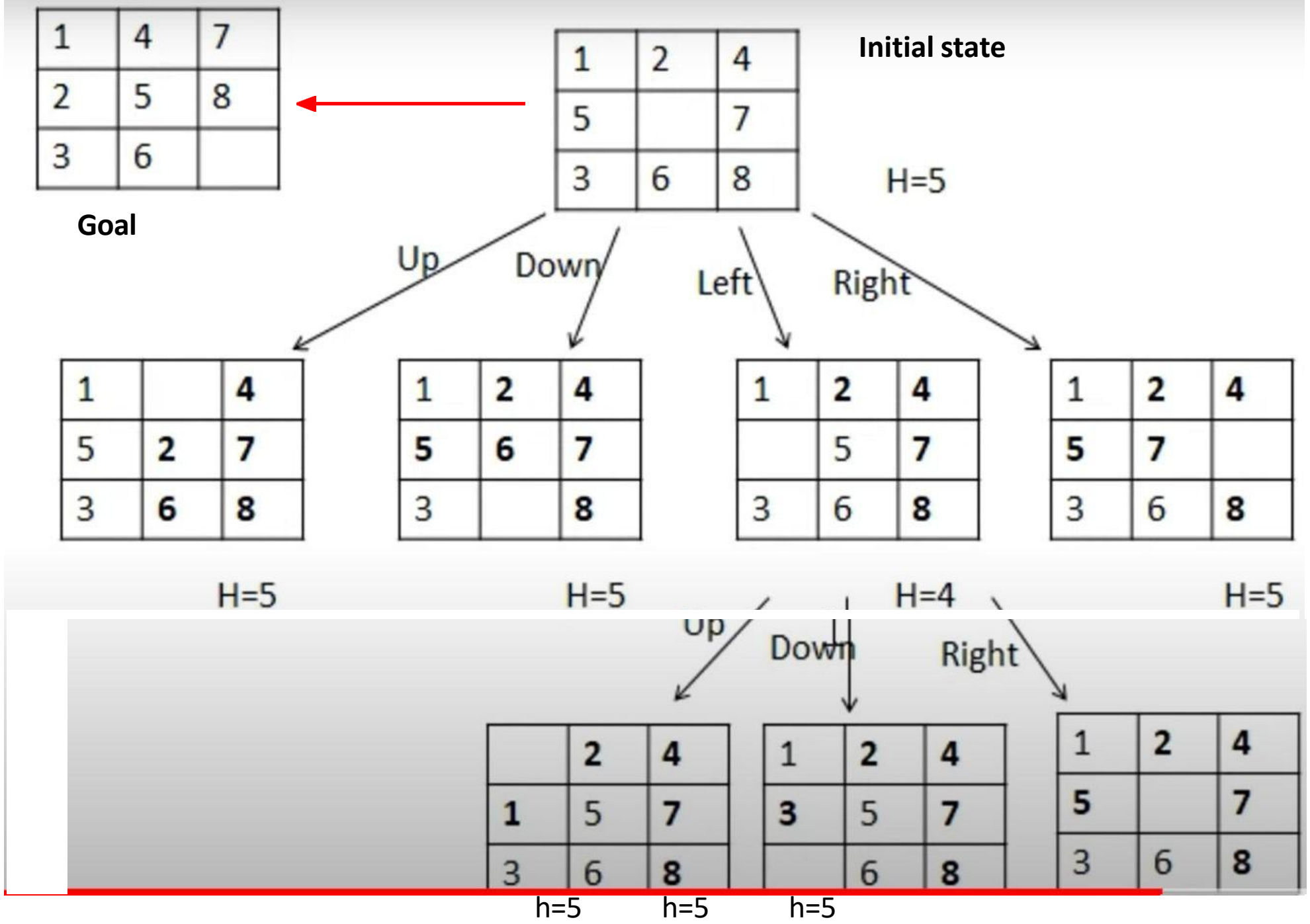
Hill Climbing

```
function HILL-CLIMBING(problem) returns a state that is a local maximum

    current ← MAKE-NODE(problem.INITIAL-STATE)
    loop do
        neighbor ← a highest-valued successor of current
        if neighbor.VALUE ≤ current.VALUE then return current.STATE
        current ← neighbor
```

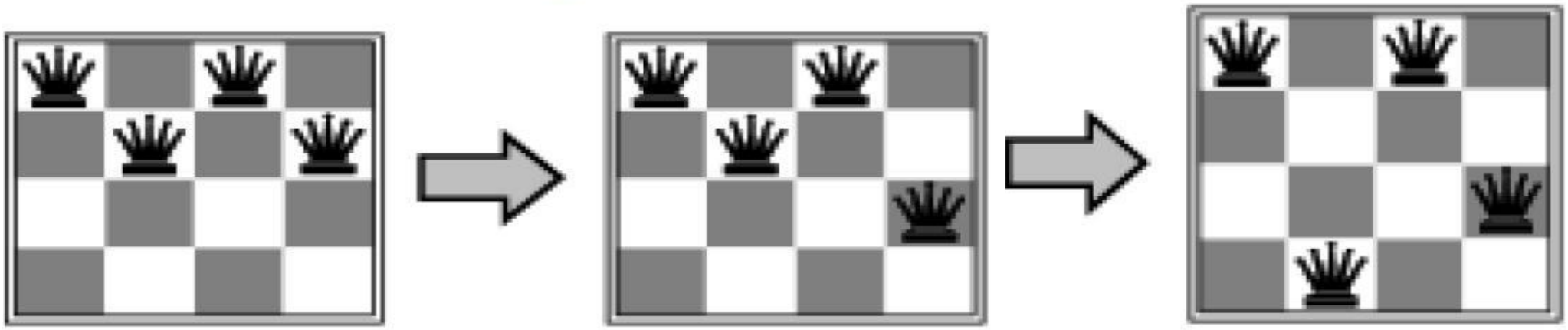
Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h.


start
3 1 2
4 5 8
6 7
h = 5
goal
1 2
3 4 5
6 7 8
6
h = 4
5
h = 0
3 1 2
4 5 8
6 7
3 1 2
4 5
6 7 8
5
h = 3
h = 1
3 1 2
4 5
6 7 8
3 1 2
4 5
6 7 8
h = 2
4


1 4 7
2 5 8
3 6
Goal
1 2 4
5 7
3 6 8
Initial state
H=5
Up
Down
Left
Right
1 4
5 2 7
3 6 8
H=5
1 2 4
5 6 7
3 8
H=5
1 2 4
5 7
3 6 8
H=4
1 2 4
5 7
3 6 8
H=5
Up
Down
Right
2 4
1 5 7
3 6 8
h=5
1 2 4
3 5 7
6 8
h=5
1 2 4
5 7
3 6 8
h=5

N-Queen Hill Climbing

- ***n*-queens problem: Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal**
- ***Good heuristic:* *h* = number of pairs of queens that are attacking each other**

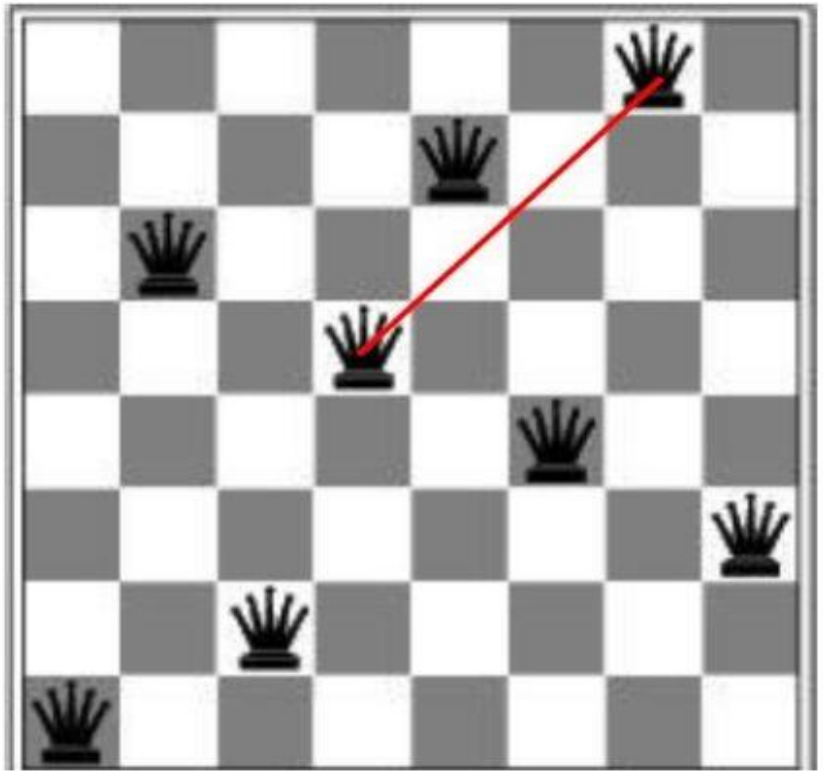


h=5 *h=3* *(for illustration)* *h=1*

8-Queen Hill Climbing



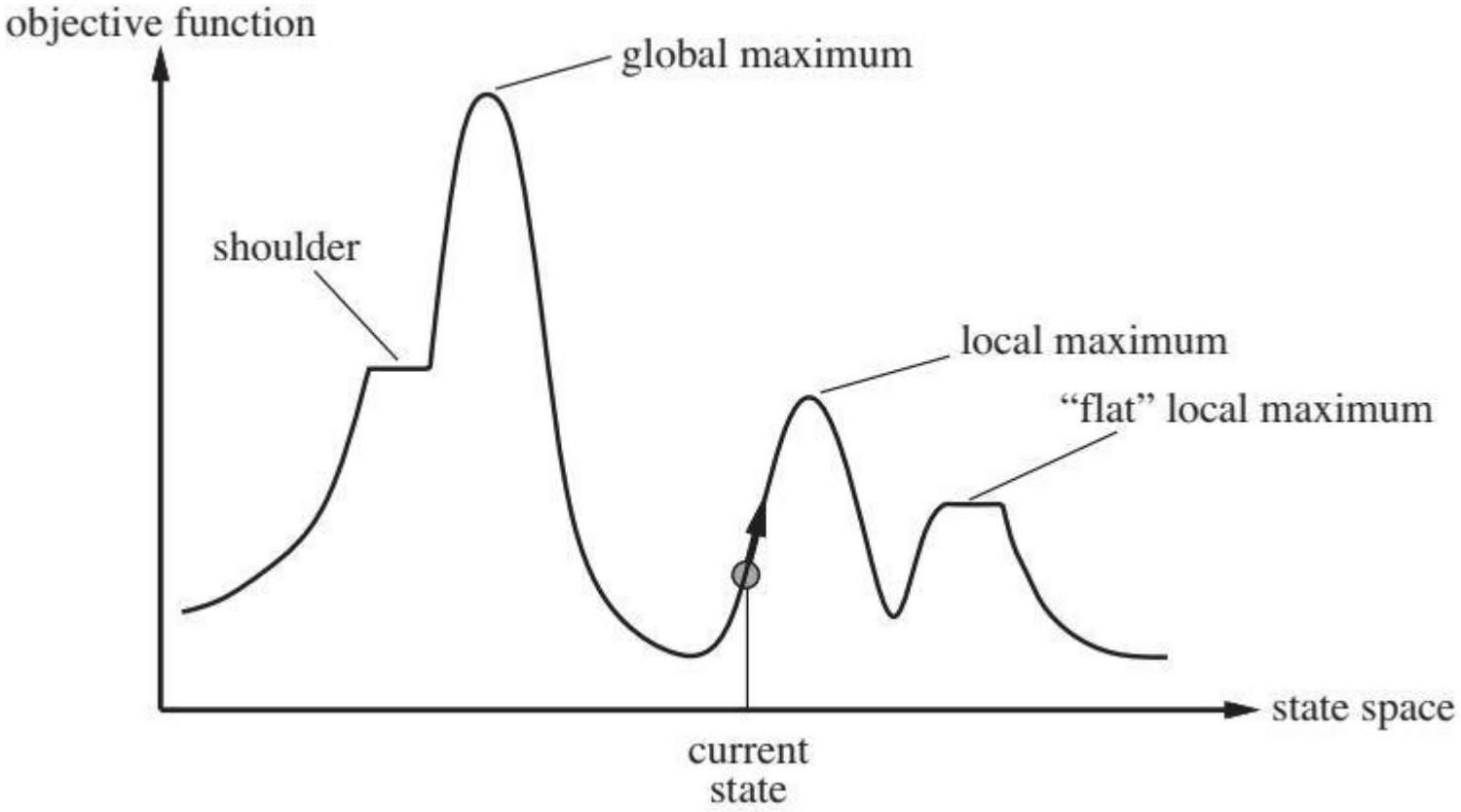
A state with h=17 and the h-value for each possible successor



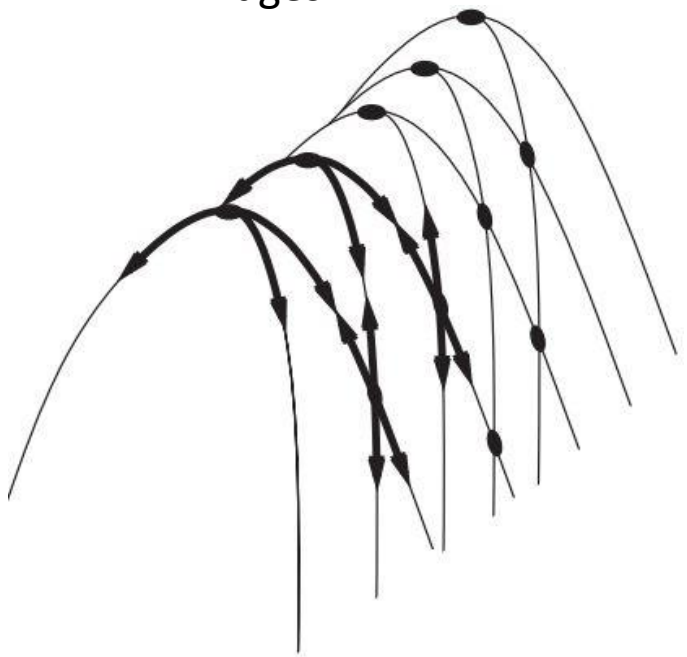
A local minimum of h in the 8-queens state space (h=1).

h = number of pairs of queens that are attacking each other

Limitations of Hill Climbing



Ridges



Limitations of Hill Climbing

- ***Local maximum :*** It is a state which is better than its neighboring state however there exists a state which is better than it(global maximum). This state is better because here value of objective function is higher than its neighbors.
- ■ ***Plateua/flat local maximum :*** It is a flat region of state space where neighboring states have the same value. So these are the flat points where all successor values are same.
- ■ ***Ridge :*** It is region which is higher than its neighbors but itself has a slope. It is a special kind of local maximum. So it’s a situation where all successors value will be same within intervals

Solutions of Limitations of Hill Climbing

- ***Stochastic hill climbing*** -- chooses at Random uphill move with certain probability
- ***First-choice hill climbing*** -- generating successors randomly until one is generated that is better than the current state.
- ***Random-restart hill*** -- “If at first you don’t succeed, try, try again.” It conducts a series of hill-climbing searches from randomly generated initial states